MODULE-3

Cryptography and Hyperledger

Dr. Vanitha M, Professor, SCORE,VIT,VELLORE

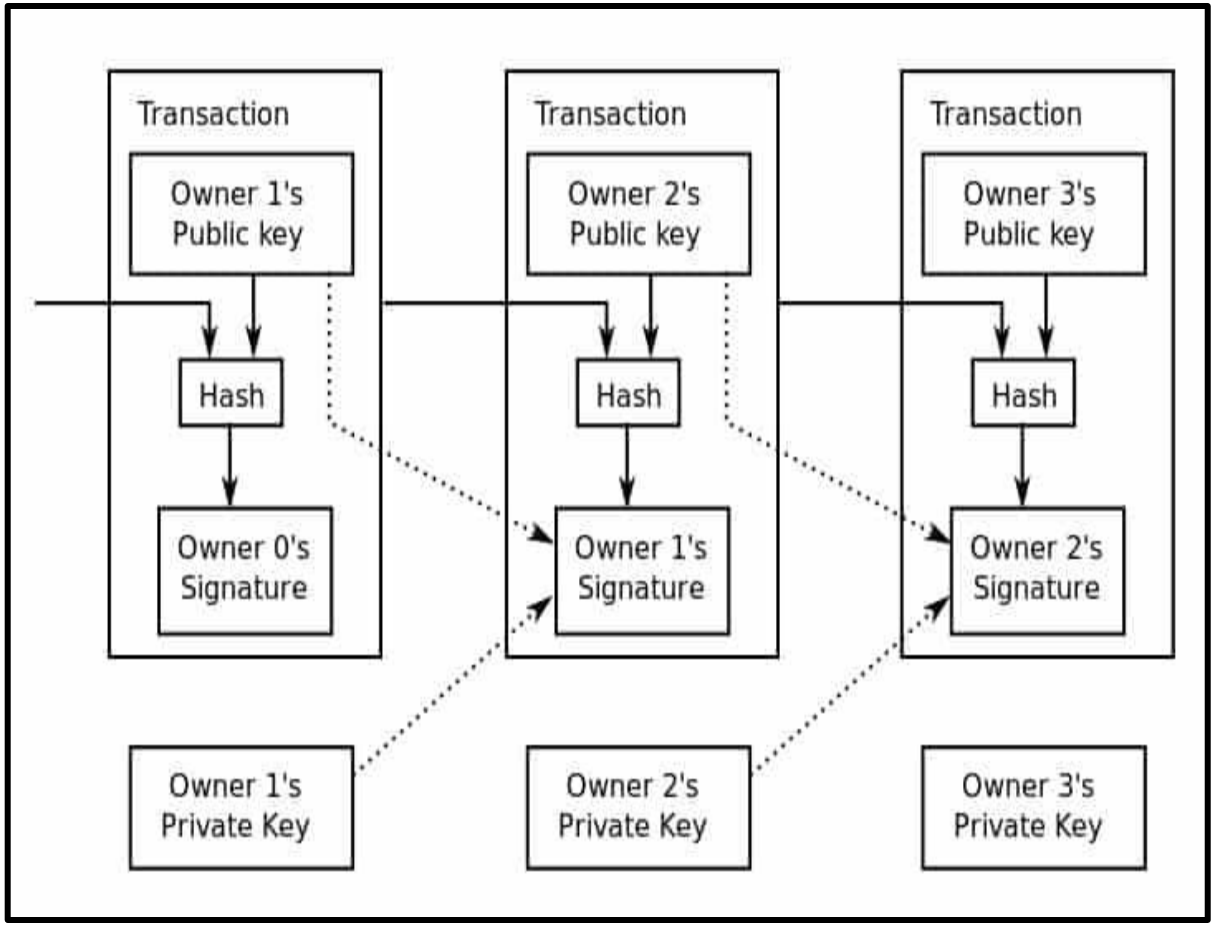
Cryptography

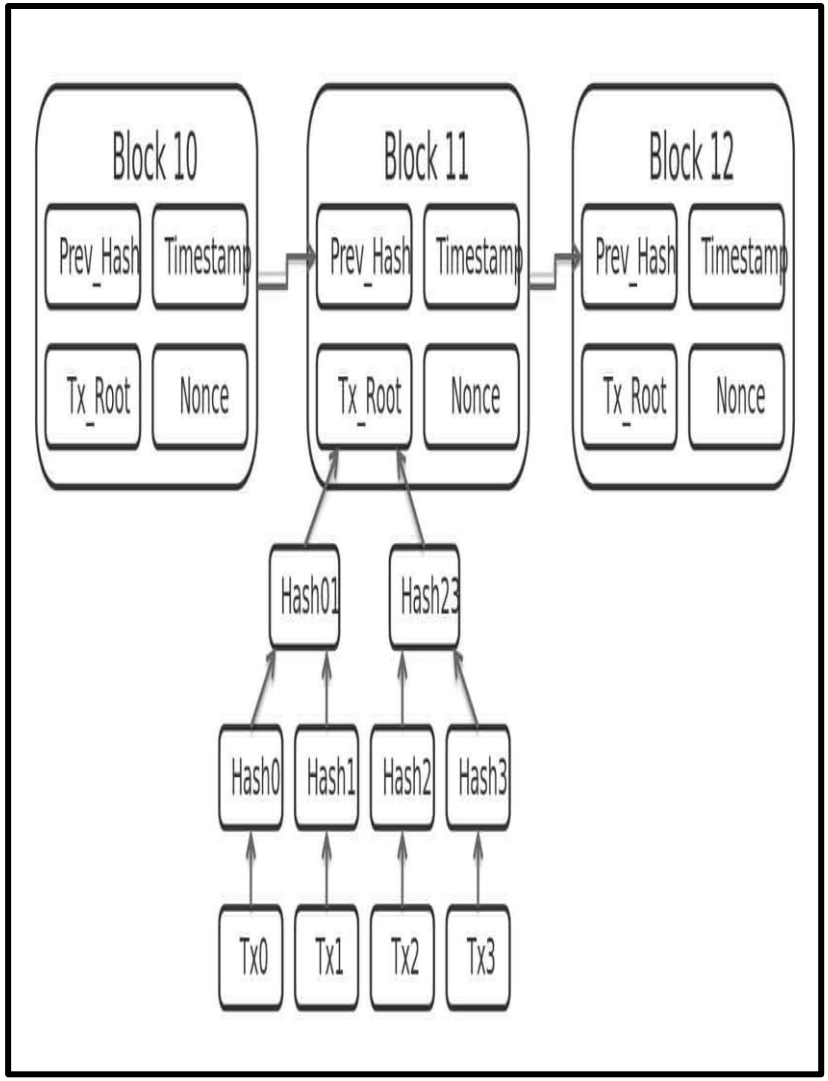
- Cryptography in blockchain refers to the use of **mathematical algorithms** to protect the data and transactions recorded on the blockchain.
- These cryptographic techniques ensure that **data is encrypted, securely transmitted, and verified by the participants** in a blockchain network, making it virtually impossible for malicious actors to alter or tamper with blockchain records.

The Role of Cryptography in Blockchain Security

- Transactions are secure and cannot be tampered with
- User identities are protected, maintaining privacy
- Data stored on the blockchain is immutable, ensuring trust

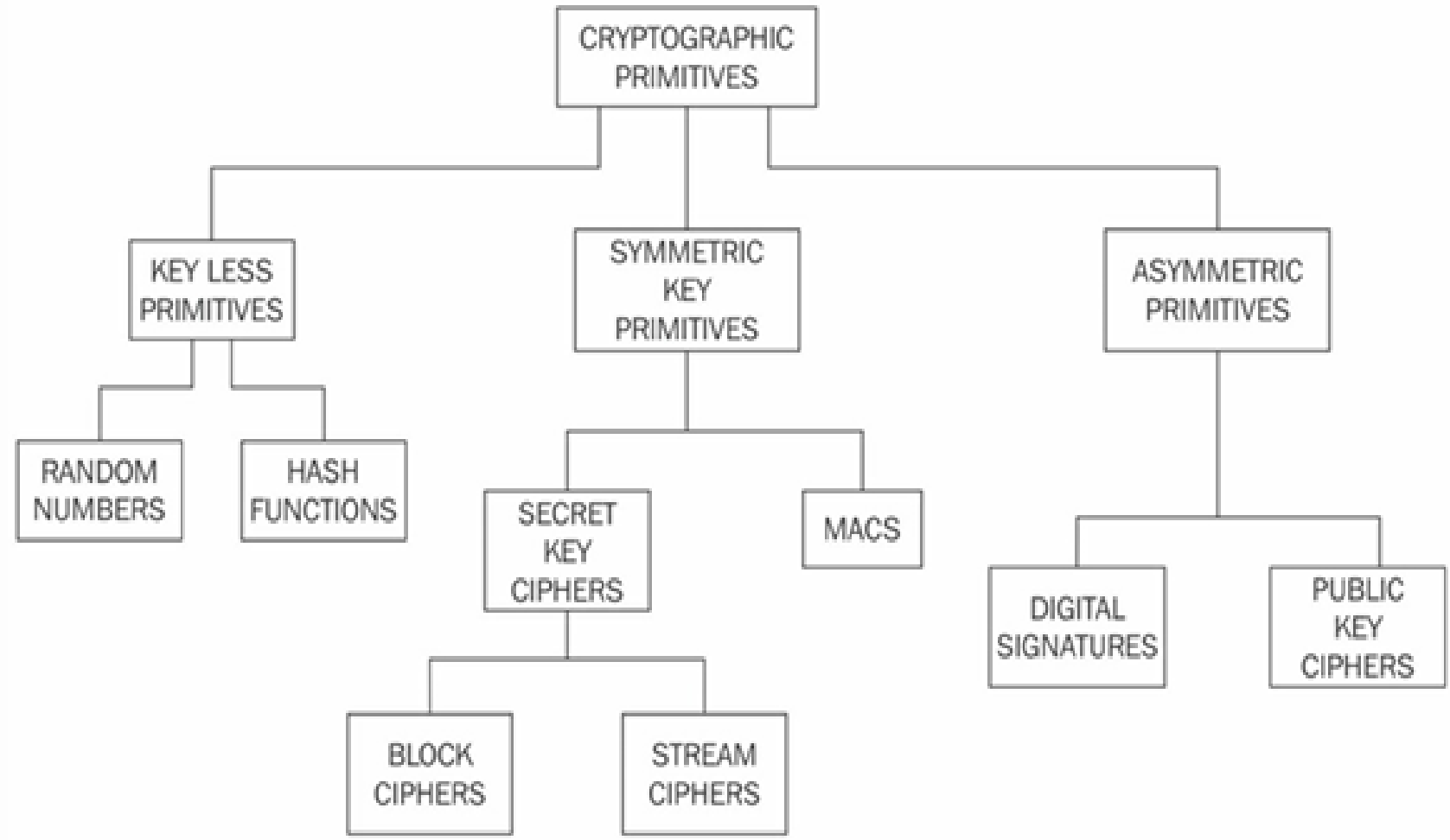
Cryptography and Blockchain





Cryptographic Primitives

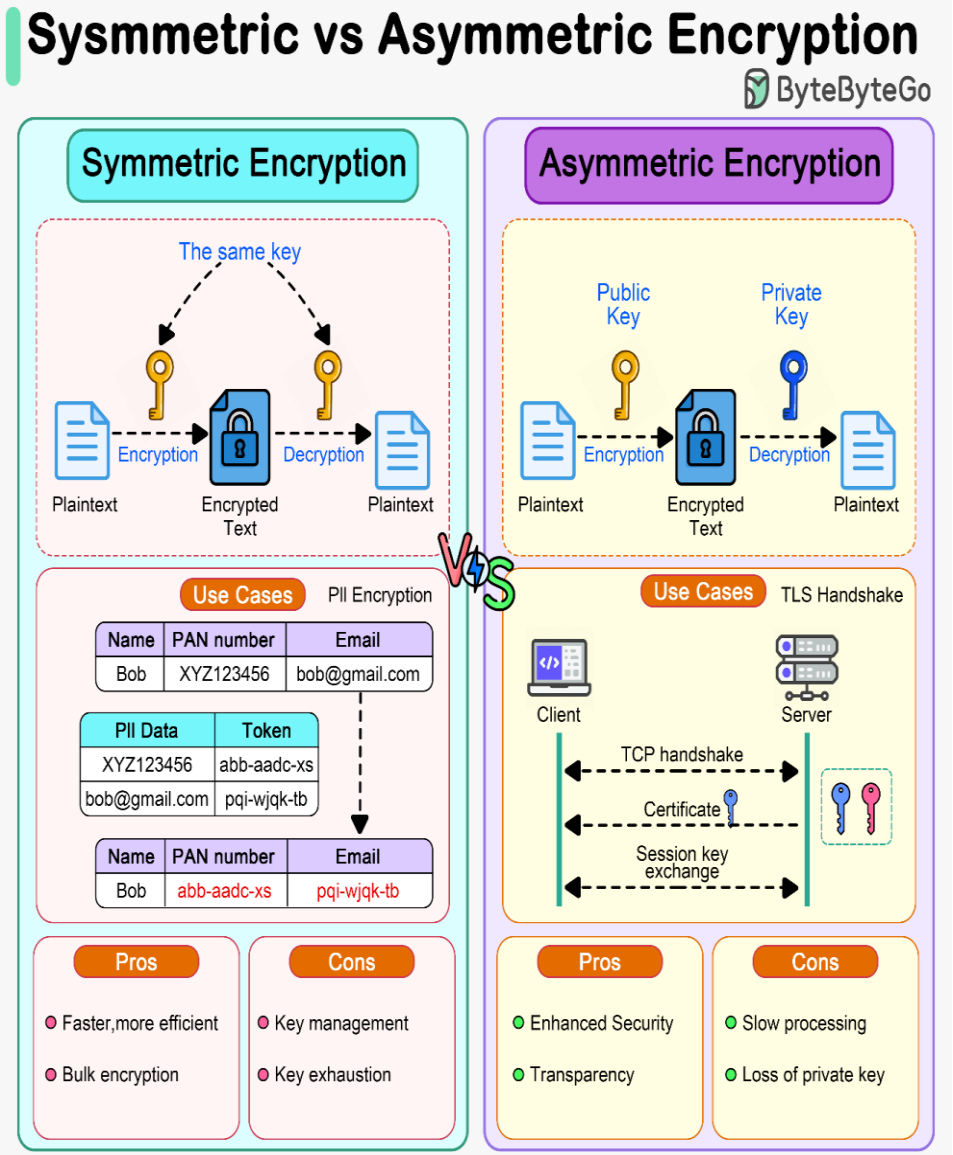
- Cryptographic primitives are the basic building blocks of a security protocol or system.
- A security protocol is a set of steps taken to achieve the required security goals by utilizing appropriate security mechanisms.
- Various types of security protocols are in use, such as authentication protocols, non-repudiation protocols, and key management protocols.



Cryptographic primitives

Symmetric cryptography

No	Symmetric Key Cryptography	Asymmetric Key Cryptography
1	Single key is used for encryption and decryption.	Two different key is used, one for encryption and other for decryption.
2	It is also called secret key or private key cryptography.	It is also called public key cryptography or conventional cryptographic system.
3	It is faster than asymmetric key cryptography.	It is slower than symmetric key cryptography.
4	It uses less resource in compare to asymmetric key cryptography.	It uses more resource in compare to symmetric key cryptography.
5	For encryption of large message symmetric key cryptography is used.	In asymmetric key cryptography plain text and cipher text treated as integer number.
6	For example: DES, AES and BLOWFISH	For example, RSA and Diffie-Hellman Key exchange.
7	Mathematically it is represented as $C = E(K, P)$ and $P = D(K, C)$ i.e., C = Cipher text, E = Encryption, D = Decryption, P = Plain text, K =Secret Key	Mathematically it is represented as $C = E(Pu(R), P)$ and $P = D(Pr(R), C)$ i.e., C = Cipher text, E = Encryption, D = Decryption, P = Plain text, Pr(R) =Private Key of receiver, Pu(R) = Public key of receiver.



Symmetric cryptography

- **Symmetric cryptography** is a class of cryptographic techniques where the **same secret key** is used for both encryption and decryption. Thus, it is also known as shared key cryptography.
  - Plaintext + Secret Key → Ciphertext
  - Ciphertext + Secret Key → Plaintext
- The key must be established or agreed upon before the data exchange occurs between the communicating parties. This is the reason it is also called secret key cryptography.
- There are two types of symmetric ciphers: stream ciphers and block ciphers.
- Data Encryption Standard (DES) and Advanced Encryption Standard (AES) are typical examples of block ciphers, whereas RC4 and A5 are commonly used stream ciphers

Stream Cipher and Block Cipher

	Stream Cipher	Block Cipher
Length	Bit or Byte	Block Size - 64 bits, 128 bits
Design	Complex	Simple
Principle	Confusion	Confusion and Diffusion
Speed	Faster	Slower
Encryption	CFB (Cipher Feedback) and OFB (Output Feedback)	Electronic Code Book (ECB) and Cipher Block Chaining (CBC)
Decryption	XOR	Reverse of encryption
Example	Vernam Cipher	DES, AES

nesoacademy.org

Stream ciphers

- **Stream ciphers** are a class of symmetric-key encryption algorithms that encrypt data **one bit or byte at a time** by combining the plaintext with a pseudorandom **keystream**.

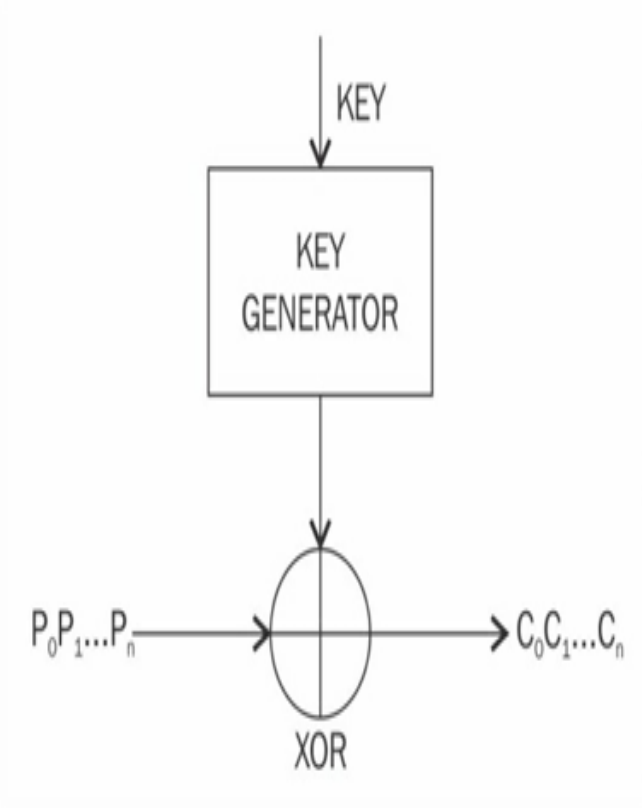
- **Synchronous stream** ciphers are those where the keystream is dependent only on the key

- **Asynchronous stream** ciphers have a keystream that is also dependent on the encrypted data
  - Keystream $\oplus$ Plaintext = Ciphertext
  - Keystream $\oplus$ Ciphertext = Plaintext

- A stream cipher generates keystream from a secret key (an nonce) and XORs it with plaintext t produce ciphertext.

Plaintext ⊕ Keystream = Ciphertext

Ciphertext ⊕ Keystream = Plaintext



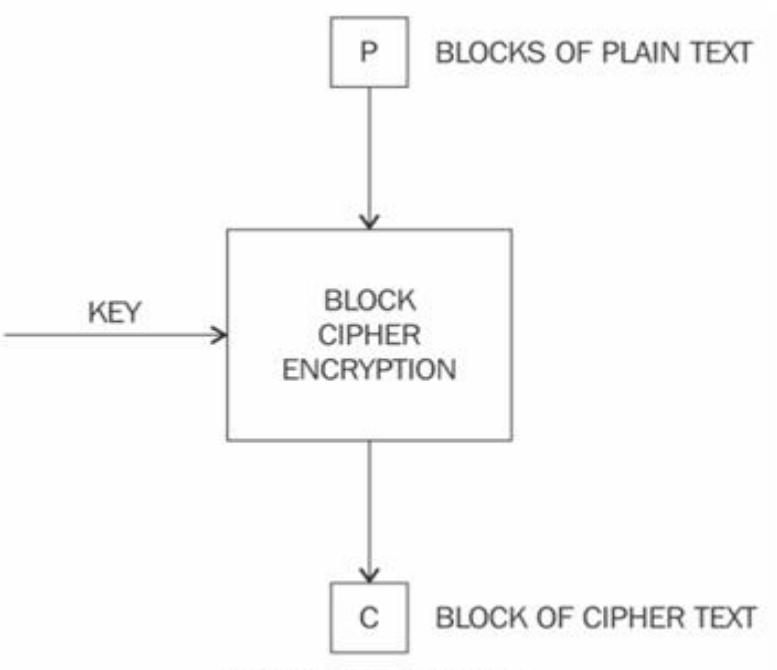
Operation of a stream cipher

Block ciphers

- A block cipher takes a fixed-size block of plaintext (e.g., 128 bits) and a secret key, and transforms it into a block of ciphertext.
  - Plaintext block + Key → Ciphertext block
  - Ciphertext block + Key → Plaintext block

- Block ciphers are generally built using a design strategy known as a Feistel cipher.
- Recent block ciphers, such as AES (Rijndael) have been built using a combination of substitution and permutation called a Substitution-Permutation Network (SPN).

- A key advantage of using a Feistel cipher is that encryption and decryption operations are almost identical and only require a reversal of the encryption process to achieve decryption.
- DES is a prime example of Feistel-based ciphers



Simplified operation of a block cipher

- Various modes of operation for block ciphers are Electronic Code Book (ECB),
- Cipher Block Chaining (CBC),
- Output Feedback (OFB) mode, and
- Counter (CTR) mode.

- These modes are used to specify the way in which an encryption function is applied to the plaintext.
-

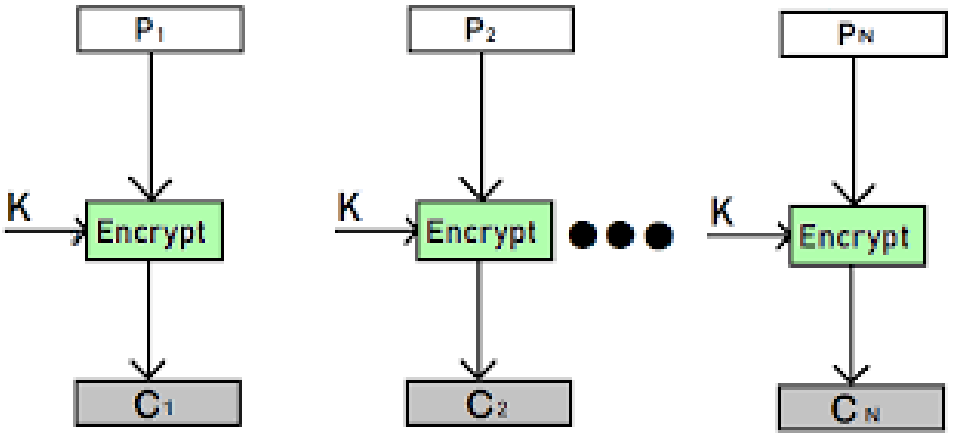
Block encryption mode

- In block encryption mode, the plaintext is divided into blocks of fixed length depending on the type of cipher used.
- Then the encryption function is applied to each block

Electronic Code Book

- **ECB mode** is the **simplest mode of operation** for block ciphers, where **each plaintext block is encrypted independently** using the same secret key.

- **Working of ECB Mode**
  - P1 ——AES——▶ C1
  - P2 ——AES——▶ C2
  - P3 ——AES——▶ C3

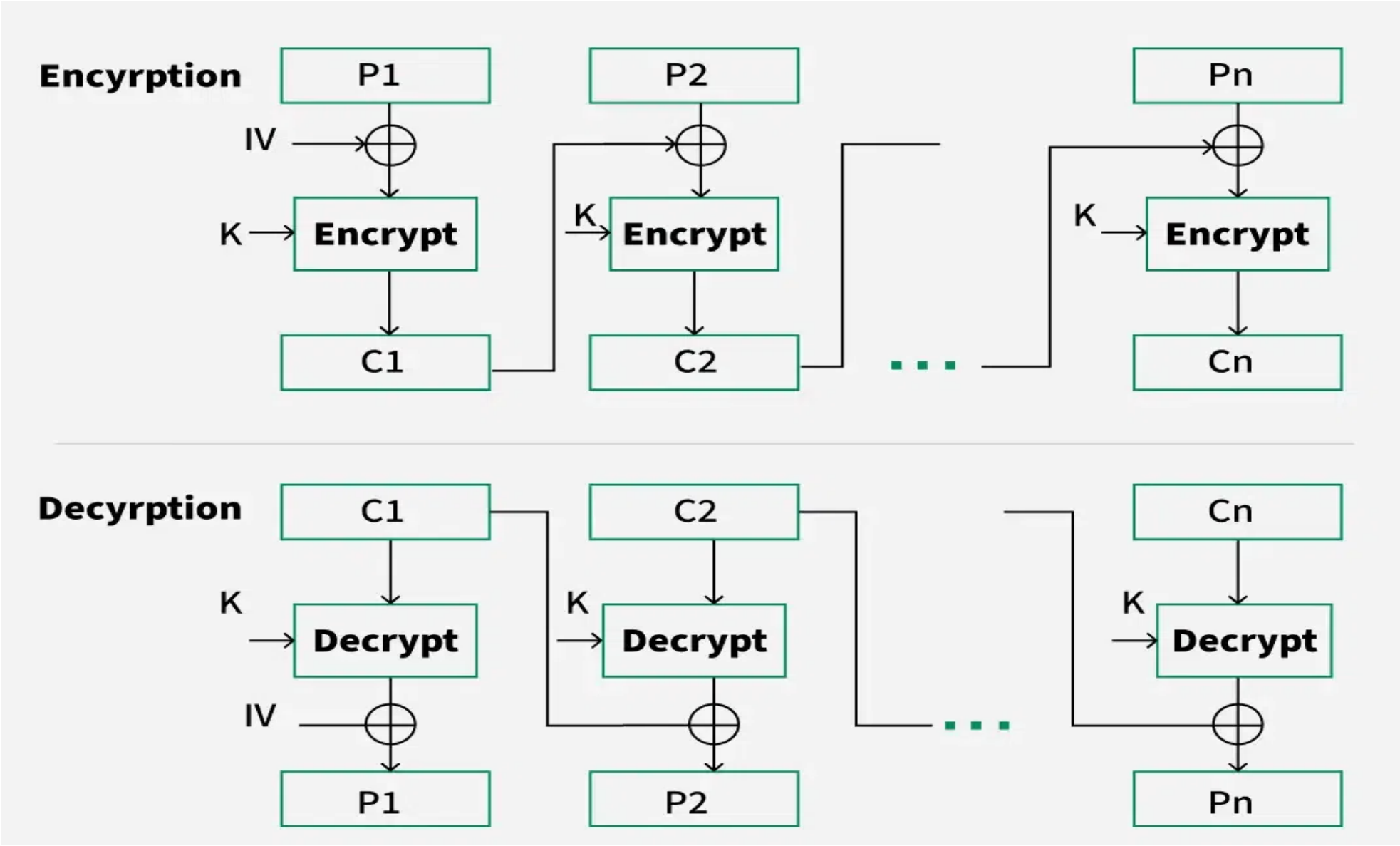


- **Advantages of using ECB**
- Parallel encryption of blocks of bits is possible, thus it is a faster way of encryption.
- Simple way of the block cipher.

- **Disadvantages of using ECB**
- Prone to cryptanalysis since there is a direct relationship between plaintext and ciphertext.
- Identical plaintext blocks produce identical ciphertext blocks, which can reveal patterns.

Cipher Block Chaining

- In Cipher Block Chaining (CBC) mode, each block of plaintext is XOR'd with the previously-encrypted block.
- CBC mode uses the Initialization Vector (IV) to encrypt the first block. It is recommended that the IV be randomly chosen

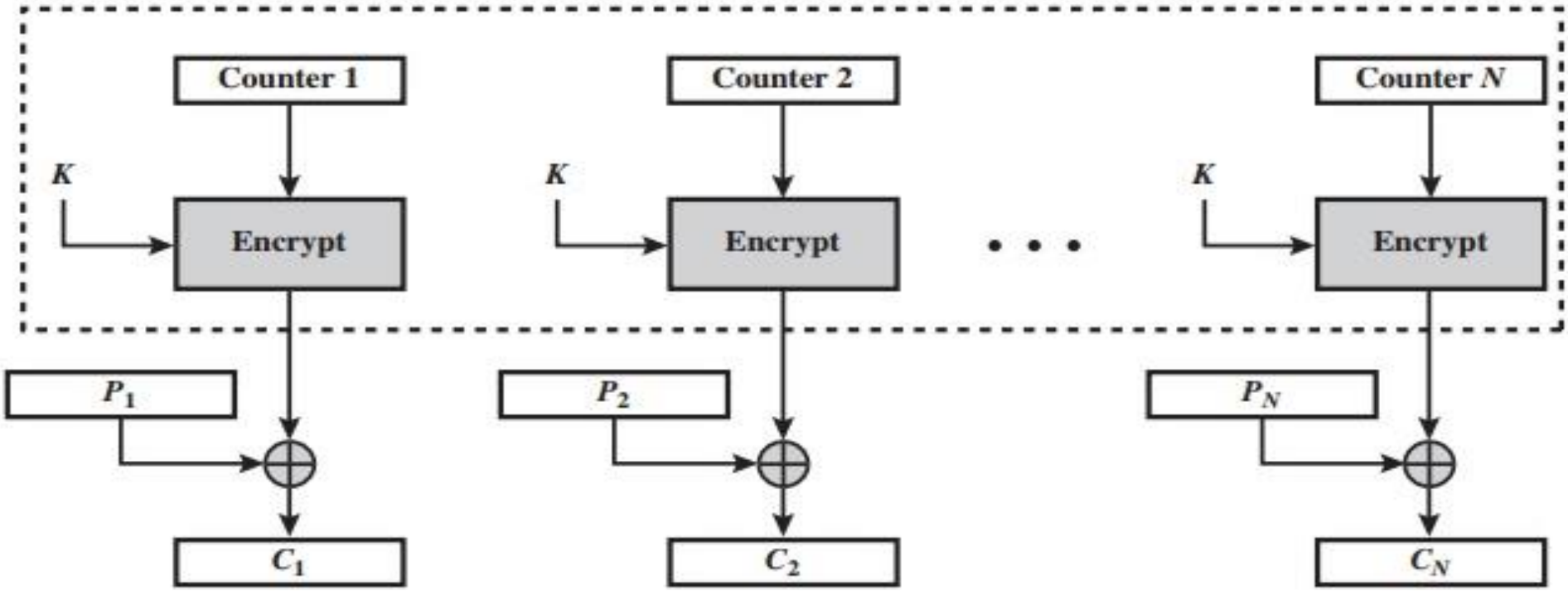

Encyrption
P1
P2
Pn
IV
K
Encrypt
K
Encrypt
K
Encrypt
C1
C2
Cn
Decyrption
C1
C2
Cn
K
Decrypt
K
Decrypt
K
Decrypt
IV
P1
P2
Pn

- **Advantages of CBC**
- CBC works well for input greater than *b* bits.
- CBC is a good authentication mechanism.
- Better resistive nature towards cryptanalysis than ECB.
- More secure than ECB as it hides patterns.

- **Disadvantages of CBC**
- Requires the previous ciphertext block for encryption and decryption, making parallel processing difficult.

Counter Mode (CTR)

- Counter Mode (CTR) is a block cipher mode of operation that turns a block cipher into a stream cipher by encrypting a sequence of incrementing values (counters) to generate a key stream, which is then XORed with plaintext blocks.
- It enables parallel encryption/decryption, high-speed performance, and random access to data, often used in network security.



(a) Encryption

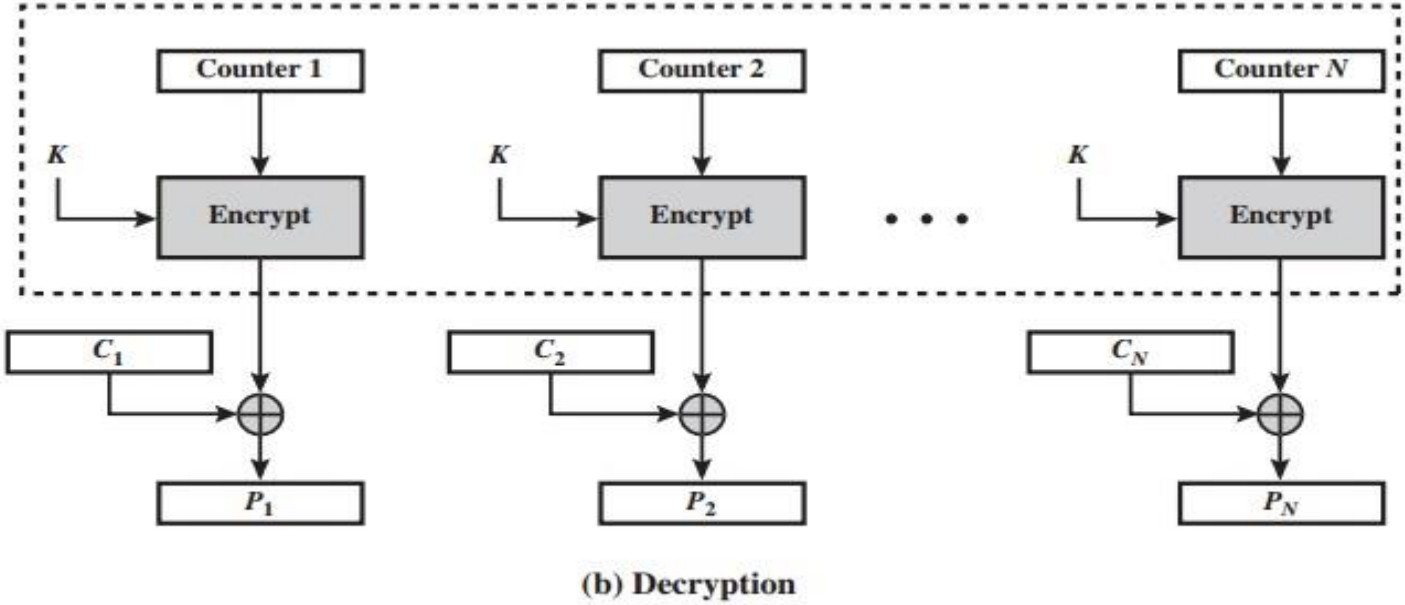
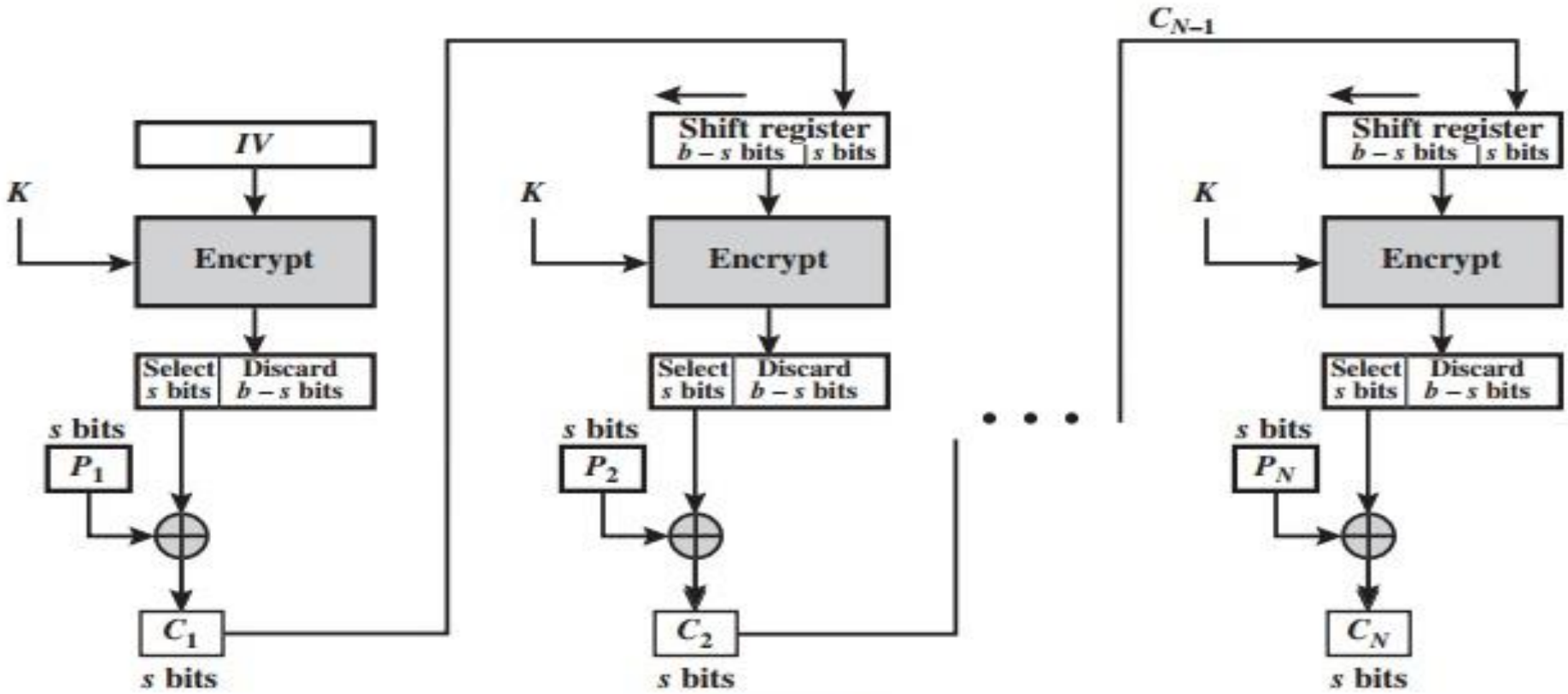


Figure 6.7 Counter (CTR) Mode

Advantages of Counter

- Since there is a different counter value for each block, the direct plaintext and ciphertext relationship is avoided. This means that the same plain text can map to different ciphertext.
- Parallel execution of encryption is possible as outputs from previous stages are not chained as in the case of CBC.
- **Disadvantages of Counter**
- The fact that CTR mode requires a **synchronous counter at both the transmitter and the receiver** is a severe drawback. The recovery of plaintext is inaccurate when synchronization is lost.

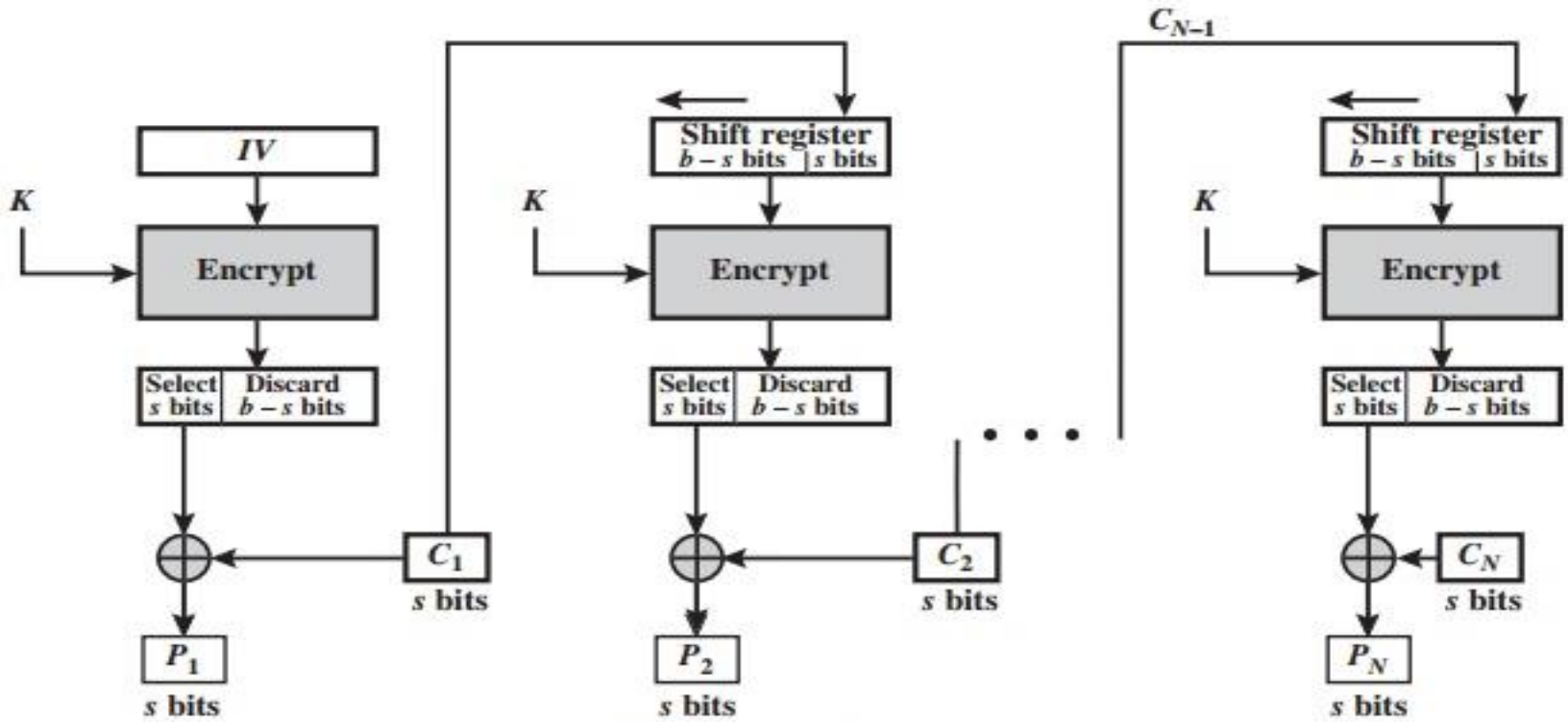
Cipher Feedback (CFB) mode



(a) Encryption

- In this mode the cipher is given as feedback to the next block of encryption with some new specifications: first, an initial vector IV is used for first encryption and output bits are divided as a set of *s* and *b-s* bits.
- The left-hand side *s* bits are selected along with plaintext bits to which an XOR operation is applied.
- The result is given as input to a shift register having b-s bits to lhs, s bits to rhs and the process continues.
- The encryption and decryption process for the same is shown below, both of them use encryption algorithms.

- The *cipher feedback* (CFB) mode, in its simplest form uses the entire output of the block cipher.
- In this variation, it is very similar to CBC, turning a block cipher into a self-synchronizing stream cipher.
- CFB decryption in this variation is almost identical to CBC encryption performed in reverse:



(b) Decryption

Figure 6.5 s-bit Cipher Feedback (CFB) Mode

CFB	$I_1 = IV$ $I_j = \mathrm{LSB}_{b-s}(I_{j-1}) \parallel C_{j-1} \quad j = 2, \ldots, N$ $O_j = \mathrm{E}(K, I_j) \quad j = 1, \ldots, N$ $C_j = P_j \oplus \mathrm{MSB}_s(O_j) \quad j = 1, \ldots, N$	$I_1 = IV$ $I_j = \mathrm{LSB}_{b-s}(I_{j-1}) \parallel C_{j-1} \quad j = 2, \ldots, N$ $O_j = \mathrm{E}(K, I_j) \quad j = 1, \ldots, N$ $P_j = C_j \oplus \mathrm{MSB}_s(O_j) \quad j = 1, \ldots, N$

- **Advantages of CFB**
- Since, there is some data loss due to the use of shift register, thus it is difficult for applying cryptanalysis.
- Can handle data streams of any size.

- **Disadvantages of using CFB**
- The drawbacks of CFB are the same as those of CBC mode. Both block losses and concurrent encryption of several blocks are not supported by the encryption. Decryption, however, is parallelizable and loss-tolerant.
- Slightly more complex and can propagate errors.

Keystream generation mode

- In cryptography, a keystream generation mode refers to the process of using a block or stream cipher algorithm to produce a continuous sequence of pseudorandom bits (the **keystream**).
- This keystream is subsequently XORed with plaintext to create ciphertext

Message authentication modes

- Message authentication modes ensure that a message originates from a legitimate sender and has not been tampered with.
- Unlike keystream generation, which focuses on secrecy, these modes focus on **integrity** and **authenticity**
- In message authentication mode, a Message Authentication Code (MAC) results from an encryption function.
- The MAC is a cryptographic checksum that provides an integrity service.

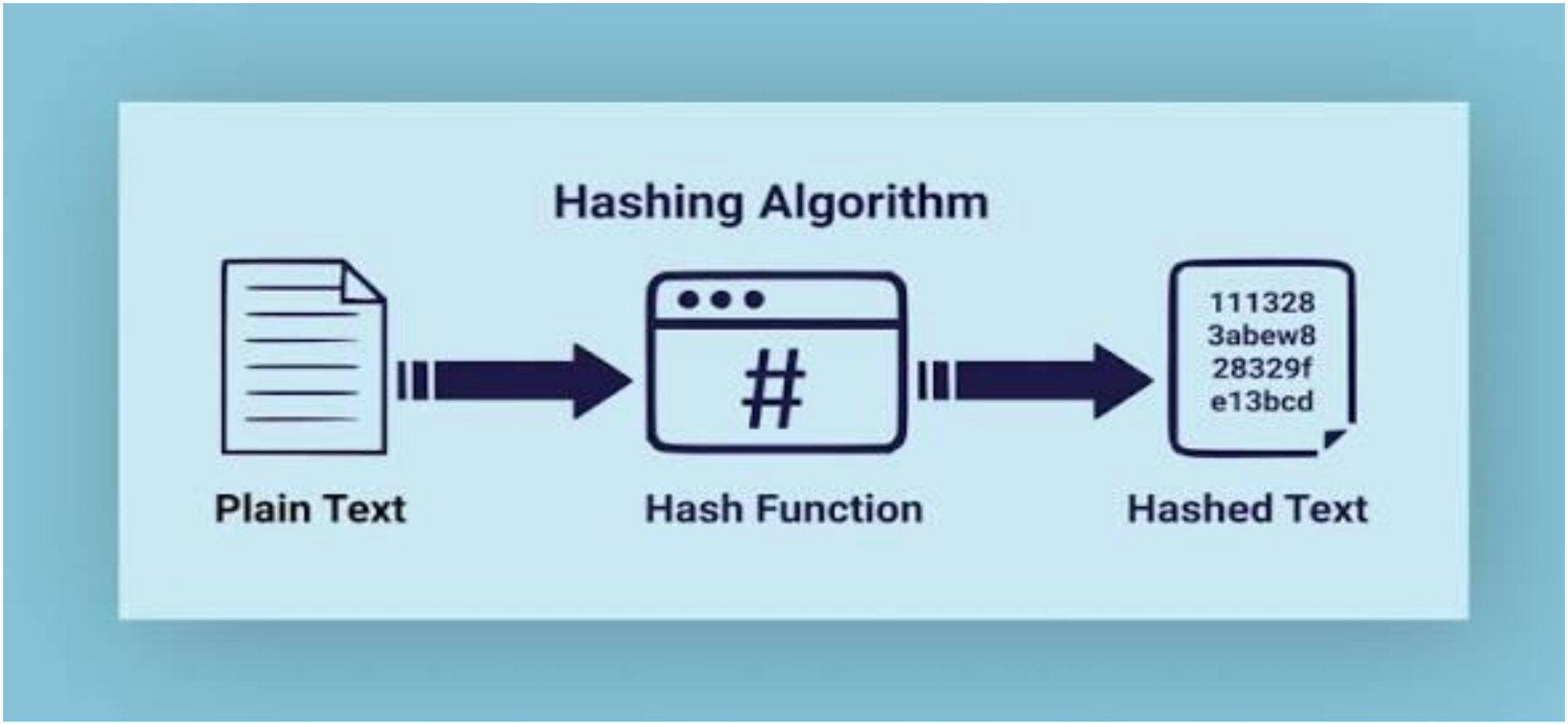
HASH FUNCTIONS

Hash functions

- **Hash function** is a mathematical algorithm that transforms an input of any size into a fixed-length string of characters, typically a hexadecimal number. Often called a **digital fingerprint** or **message digest**, it serves as a unique identifier for data.

- They are deterministic, fast, and highly resistant to collisions, where even a minor input change produces a completely different hash.

- Key applications include secure password storage, blockchain (e.g., SHA-256 in Bitcoin), digital signatures, and detecting data tampering.


Hashing Algorithm
#
111328
3abew8
28329f
e13bcd
Plain Text
Hash Function
Hashed Text

Core Properties of Cryptographic Hash Functions

- **Deterministic:** The same input always produces the exact same output hash.
- **Fixed Output Size:** Regardless of the input size (a single letter or a huge file), the output length is always constant.
- **Pre-image Resistance (One-way):** It is computationally infeasible to reverse-engineer the original input data from its hash value.
- **Avalanche Effect:** A minor modification to the input (e.g., changing a single bit) produces a drastically different hash value.
- **Collision Resistance:** It is extremely difficult to find two different inputs that produce the same output hash.
- **Fast Computation:** The function can quickly generate the hash for any given data.

Common Hashing Algorithms

Algorithm	Status	Typical Use Cases
SHA-256	**Standard**	Bitcoin, digital signatures, SSL/TLS certificates.
SHA-3	**Latest Standard**	Advanced security applications, backup to SHA-2.
BLAKE2 / BLAKE3	**Modern**	High-performance systems requiring speed and security.
SHA-1	**Deprecated**	Legacy systems (insecure due to collision vulnerabilities).
MD5	**Legacy**	Fast checksums only; broken for any security-critical use.

Where can it be implemented?

Digital Signatures

- SHA-256 is often used with public-key cryptography to create digital signatures. It allows for the verification of the integrity and authenticity of digital messages or documents.

Blockchain Technology

- SHA-256 plays a fundamental role in blockchain technology. It generates the hash values of blocks in a blockchain, ensuring the immutability and integrity of the entire chain.

Password Hashing

- In secure password storage, systems store the SHA-256 hash of the password instead of storing the actual password. This enhances security by preventing exposure to plain-text passwords in case of a data breach.

Secure Communication

- Hash functions like SHA-256 are used in secure communication protocols to ensure the integrity of transmitted data. For example, in TLS/SSL protocols, hash functions are used with digital signatures to provide secure communication over the Internet.

Certificate Authorities (CAs)

- SHA-256 is commonly used to create and verify digital certificates issued by Certificate Authorities. It helps ensure the authenticity and integrity of these certificates.

File Integrity Checking

- SHA-256 generates checksums or hash values for files. Users can then verify the integrity of files by comparing the computed and original hash.

Message Digest 5

- It is a cryptographic protocol used for authenticating messages as well as content verification and digital signatures.

- MD5 is based on a hash function that verifies that a file you sent matches the file received by the person you sent it to.

- Previously, MD5 was used for data encryption, but now it's used primarily for authentication

- A key characteristic of is to detect any alterations made to the input information, regardless of whether the modification is as minute as a single bit. This is known as the "**integrity**" of the data, which is fundamental in computer security.

- Whenever a file or communication is transmitted over the internet, its Cryptographic Hash Algorithm 5 hash is computed and sent along with it. The recipient then computes the Cryptographic Hash Algorithm 5 hash of the received information and compares it to the original hash to confirm that it hasn't been altered during transmission.

Working of the MD5 Algorithm

1. Append Padding Bits: In the first step, we add padding bits in the original message in such a way that the total length of the message is 64 bits less than the exact multiple of 512.

*Suppose we are given a message of 1000 bits. Now we have to add padding bits to the original message. Here we will add 472 padding bits to the original message. After adding the padding bits the size of the original message/output of the first step will be 1472 i.e. 64 bits less than an exact multiple of 512 (i.e. 512*3 = 1536).*

Length(original message + padding bits) = 512 * i - 64 where i = 1,2,3…

2. Append Length Bits: In this step, we add the length bit in the output of the first step in such a way that the total number of the bits is the perfect multiple of 512. Simply, here we add the 64-bit as a length bit in the output of the first step.

- **3. Initialize MD buffer:** Here, we use the 4 buffers i.e. A B, C, and D. The size of each buffer is 32 bits.
- - A = 0x67425301
- - B = 0xEDFCBA45
- - C= 0x98CBADFE
- - D = 0x13DCE476

- **4. Process Each 512-bit Block:**
- This is the most important step of the MD5 algorithm. Here, a total of 64 operations are performed in 4 rounds.
- In the 1st round, 16 operations will be performed,
- 2nd round 16 operations will be performed,
- 3rd round 16 operations will be performed,
- 4th round, 16 operations will be performed.
- **5: Final Result (The Hash)**
- After all 64 steps, the final contents of the four buffers are concatenated to produce the 128-bit hash

Rounds	Process P
1	(b AND c) OR ((NOT b) AND (d))
2	(b AND d) OR (c AND (NOT d))
3	b OR c OR d
4	c OR (b OR (NOT d))

Process P Operation

Alternatives to MD5 in Modern Cryptography

- **SHA-256 (Secure Hash Algorithm 256-bit)** is a part of the SHA-2 family that produces a 256-bit hash value. It is widely used in blockchain, SSL certificates, digital signatures, and certificates.
- **SHA-3 offers an alternative to SHA-2** with a different internal structure, making it more resilient to certain attacks. While SHA-2 is still more common, SHA-3 is used for secure applications requiring the highest cryptographic standards.
- **RIPEMD-160** produces a 160-bit hash value, providing better security than MD5 but not as widely used as SHA-2. It is sometimes used in digital signatures and certificates in specific security systems.
- **Whirlpool** is a cryptographic hash function designed for high security, producing a 512-bit hash output. It's used particularly in cases where very strong security is required.

SHA-256

- **SHA-256** (Secure Hash Algorithm 256-bit) is a cryptographic hash function that generates a fixed-length **256-bit (32-byte)** signature for any input data.
- Developed by the NSA National Security Agency (NSA) and the and published by NIST National Institute of Standards and Technology (NIST) in 2001, it is the industry standard for secure data verification.

Key Characteristics and Uses:

- **Irreversible:** SHA-256 is a one-way function; you cannot determine the original input from the output.
- **Collision Resistant:** It is computationally infeasible to find two different inputs that produce the same output, making it highly secure.
- **Applications:** It is used to verify file integrity, generate digital signatures, and in cryptocurrency mining (e.g., Bitcoin).
- **Avalanche Effect:** A minor change in the input (e.g., changing a single character) results in a completely different hash.

- **Example:** The SHA-256 hash of "hello" is 2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824.

How It Works

- The algorithm processes data through a specific sequence of mathematical operations:
- **Padding:** It adds bits to the original message to ensure its length is a multiple of 512 bits.
- **Parsing:** The padded message is chopped into 512-bit blocks.
- **Iteration:** Each block undergoes **64 rounds** of complex logical functions (shuffling, bitwise operations, and additions with constants).
- **Final Hash:** The results of all blocks are combined to produce the final 256-bit "fingerprint"

1.Preprocessing (Padding and Length)

- Like MD5, SHA-256 must first prepare the message so its length is a multiple of **512 bits**.
- **Padding:** A single "1" bit is added to the end of the message, followed by enough "0" bits so that the message length is exactly **64 bits less** than a multiple of 512.
- **Appending Length:** The original message length is converted into a 64-bit big-endian integer and appended to the very end.
- **Result:** The final padded message is now exactly $N \times 512$ bits long

2. Initializing 8 Hash Values

- While MD5 uses four registers, SHA-256 uses **eight 32-bit registers** (h0 through h7).
- These are initialized with the first 32 bits of the fractional parts of the **square roots** of the first 8 prime numbers (2, 3, 5, 7, 11, 13, 17, 19).

3. Message Schedule Generation

- Each 512-bit block is broken into 16 words (32 bits each).
- These are then "expanded" into a schedule of **64 words**
- The first 16 words are taken directly from the message; the remaining 48 are calculated using specific bitwise logical functions (shuffling and rotations)

4.Compression Function (The 64 Rounds)

- The algorithm processes each block through **64 rounds** of operations.
- **Round Constants (K):** Each round uses a unique 32-bit constant derived from the cube roots of the first 64 prime numbers.
- **Mathematical Operations:** In every round, logical functions (like "Choose" and "Majority"), bitwise rotations, and modular additions (2power32)mix the data from the message schedule with the eight registers.
- **The Avalanche Effect:** This complex mixing ensures that changing just one bit in the input results in an entirely different output.

5. Final Output

- After all 512-bit blocks are processed, the final values in the eight registers are concatenated to produce the **256-bit hash**.

Comparison: SHA-256 vs. MD5

Feature	MD5	SHA-256
Output Size	128 bits	256 bits
Rounds	4 rounds (64 steps)	64 rounds
Internal Registers	4 (A, B, C, D)	8 (h0 through h7)
Security	**Broken** (vulnerable to collisions)	**Highly Secure**
Primary Use	Non-critical checksums	Cryptocurrencies, SSL, Signatures

Hash Properties in Blockchain

- Cryptographic hash functions provide essential security features for blockchain:
- **Deterministic:** The same input always produces the exact same output hash.
- **Collision-Resistant:** It is computationally infeasible to find two different inputs that produce the same output hash.
- **Pre-image Resistant (One-way):** It is impossible to reverse-engineer the original data from its hash value.
- **Avalanche Effect:** A tiny, one-bit change in the input data results in a completely different, drastic change in the output hash.
- **Fixed Output Length:** Regardless of input size, the hash output is always a fixed size (e.g., 256 bits for SHA-256).

Data Structures in Blockchain

- Blockchain combines these properties with specific data structures to maintain security and structure.

- **Hash Pointers (Linked Lists):** Unlike regular pointers that indicate where data is stored, hash pointers also store the hash of that data. In a blockchain, each block contains a hash pointer to the previous block, forming a chain. If any data in a block is altered, the hash pointer in the subsequent block becomes invalid, breaking the chain and exposing the tampering.

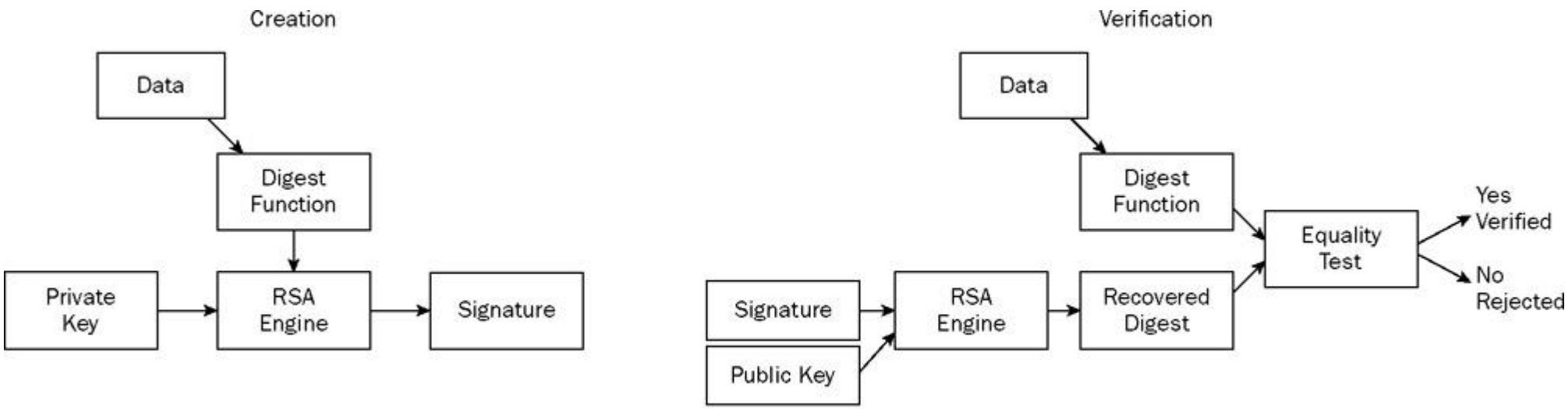
- **Merkle Trees (Binary Hash Trees):** Used to efficiently summarize and verify the integrity of transactions within a block. It is a tree structure where each leaf node is a hash of transaction data, and each non-leaf node is a hash of its children. This allows for quick verification of a single transaction without downloading the entire blockchain.

- **Block Structure:** A block consists of a header (containing the hash of the previous block, timestamp, nonce, and Merkle root) and the body (containing the transactions).

Digital signatures

- Digital signatures are the cryptographic backbone of blockchain technology, ensuring the authenticity, integrity, and non-repudiation of transactions.
- They use asymmetric cryptography (a pair of private and public keys)
- This mechanism eliminates the need for intermediaries and prevents tampering.
- Digital signatures are used in blockchain where the transactions are digitally signed by senders using their private key before broadcasting the transaction to the network.
- This digital signing, proves they are the rightful owner of the asset, for example, bitcoins.
- These transactions are verified again by other nodes on the network to ensure that the funds indeed belong to the node (user) who claims to be the owner.

RSA digital signature algorithm



RSA Signature Processes

- **Sender creates a hash** of the message.
- **Private key encrypts the hash**, creating a **digital signature**.
- Message + signature are sent over the network.
- Receiver **decrypts the signature** using the sender’s **public key**.
- Receiver **re-hashes the message**.
- If both hashes match:
- Sender is authentic
- Message is unchanged
- Sender cannot deny sending it

RSA digital signature algorithm

1. Calculate the hash value of the data packet: This will provide the data integrity guarantee as the hash can be computed at the receiver's end again and matched with the original hash to check whether the data has been modified in transit.

1. Signs the hash value with the signer's private key: As only the signer has the private key, the authenticity of the signature and the signed data is ensured.

Digital signatures properties

- Authenticity means that the digital signatures are verifiable by a receiving party.
- The unforgeability property ensures that only the sender of the message can use the signing functionality using the private key. In other words, no one else can produce the signed message produced by a legitimate sender.
- Nonreusability means that the digital signature cannot be separated from a message and used again for another message